Software Architecture and Design Specification

Project: Task Manager Application
Version: 1.1
Authors: Task Manager Project Team
Date: 15-10-2026
Status: Draft — Updated for Sprint 1

Revision History

Version	Date	Author	Change Summary
1.0	30-09-2026	Project Team	Initial architecture and design
1.1	15-10-2026	Project Team	Updated technology stack, architecture, implementation status, backend design and testing details

Approvals

Role	Name	Signature / Date
Architect / Tech Lead		
Project Guide		
QA Lead		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Task Manager Application, a tool to organise and track work progress.

1.2 Scope

Covers user services: account management, task and project management, assignment, search/filter, Kanban board and dashboard, reminders, comments, activity history, CSV export and user administration.

1.3 Audience

Developers, QA engineers, security reviewers, instructors and maintenance teams.

1.4 Definitions

SPA (Single-Page Application), REST, JWT, RBAC, ORM, ADR (Architecture Decision Record), STRIDE, TLS, API, CRUD.

2. Document Overview

2.1 How to use this document

Provides the architectural deliverables: component (UML) diagram, architecture decision records, technology stack, security threat model, UML sequence diagrams and API design. It should be read together with the SRS and the Test Plan.

2.2 Related Documents

Software Requirements Specification (SRS v1.0), Software Test Plan (STP v1.0), Requirements Traceability Matrix (RTM, SRS Section 8).

2.3 Implementation Status Convention

This version distinguishes what exists today from what is intended. **Implemented** means the item exists in the project code. **Planned** means it is part of the intended architecture but has not yet been built or tested. Planned items must not be read as working functionality.

3. Architecture

3.1 Goals & Constraints

- **Goals:** simple and fast task handling, secure user data, 99.5% availability, easy maintenance, and support for 1,000 concurrent users.
- **Constraints:** open-source stack, browser-only client, OWASP-aligned security practices, delivery within the academic timeline.

3.2 Stakeholders & Concerns

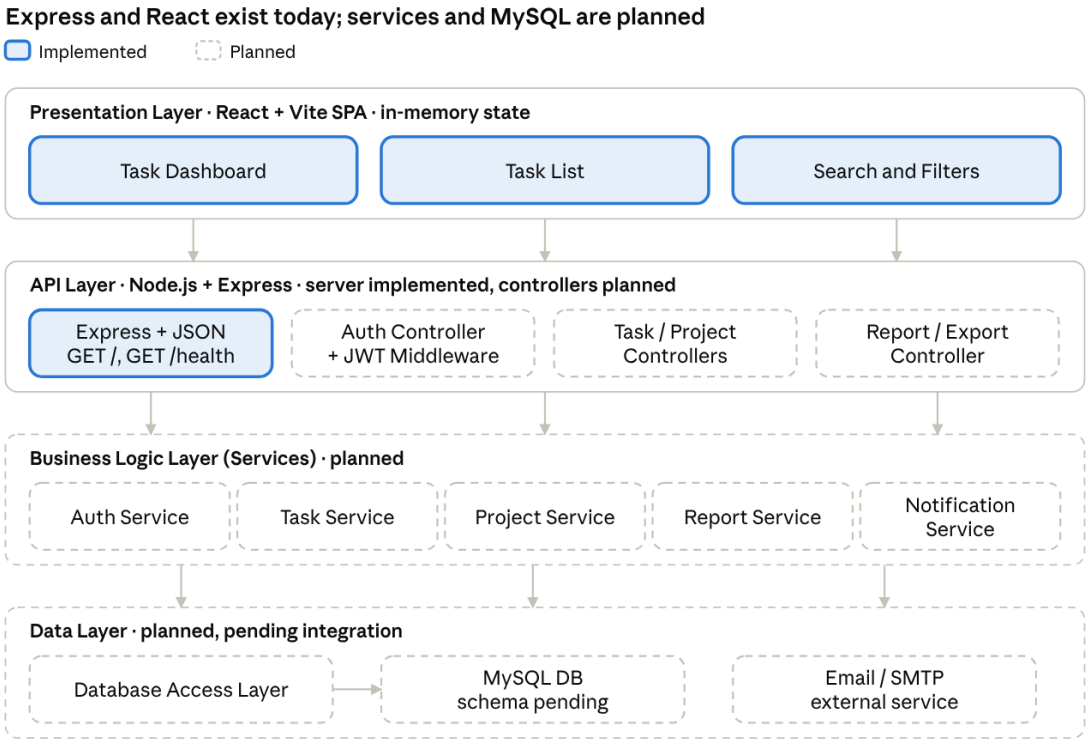
Stakeholder	Concerns
End users (Members)	Usability, speed, reliable reminders, data privacy
Project Owners / Team Leads	Clear progress tracking, access control
Administrators / Ops	Maintainability, monitoring, backups
Developers	Modularity, testability, clear interfaces
QA / Evaluators	Traceability to requirements, testability

3.3 Component (UML) Diagram

The system is organised into four layers. Each layer only calls the layer directly beneath it. The Presentation Layer (React + Vite) and the basic Express server are implemented. The Business Logic

Layer and the Data Layer (database access layer and MySQL) are planned; the current Express server is implemented, but business services, database access and MySQL integration remain pending.

Layer	Contents	Status
Presentation Layer	React + Vite SPA: Task Dashboard, Task List, Search and Filters	Implemented (in-memory state)
API Layer	Node.js + Express, REST endpoints, JSON middleware	Basic server implemented; controllers planned
Business Logic Layer	Task Service, Project Service, Authentication, Validation	Planned
Data Layer	Database access layer → MySQL DB	Planned — pending integration



layered architecture · 4 layers, implemented vs planned

Solid boxes exist today; dashed boxes are planned. Arrows show the intended call direction: each layer calls only the layer beneath it.

Figure 1 - Component Diagram (Layered Architecture)

3.4 Component Descriptions

Component	Updated responsibility / status
Web Client (React + Vite SPA)	React/Vite dashboard and task UI; currently implemented with in-memory task state.
Express API	Basic server, JSON middleware, root endpoint and health endpoint implemented.
Auth Controller and JWT Middleware	Planned; not yet implemented. Intended to handle register/login/reset endpoints and verify tokens and roles on protected requests.
Task / Project Controllers	Planned; task CRUD and project endpoints are pending.
Task and Project Services	Planned business logic: task rules, status transitions, assignment, comments, project membership and access checks.
Report and Notification Services	Planned dashboards, exports and reminders.
Database Access Layer	Planned; MySQL connection not yet integrated.
MySQL Database	Planned persistent storage; schema work pending.

3.5 Chosen Architecture Pattern and Rationale

A layered (n-tier) client-server architecture with a stateless REST API was chosen because it separates concerns (UI, API, business logic, data), is easy for a small team to develop and test, and scales horizontally by running several identical API instances behind a load balancer. Microservices were rejected as overly complex for the expected scale and team size.

ADR	Decision	Alternatives considered	Reason / Status
ADR-01	Layered monolithic backend	Microservices	Lower operational complexity; sufficient for scale
ADR-02	React single-page application	Server-rendered pages	Rich interactions (drag-and-drop Kanban, planned), responsive UI
ADR-03	MySQL relational database	PostgreSQL, MongoDB	MySQL supports relational data, foreign-key relationships and transactional operations needed to

ADR	Decision	Alternatives considered	Reason / Status
			manage users, projects, tasks and task assignments. **Status:** Selected; database schema and application integration pending.
ADR-04	Stateless JWT authentication	Server-side sessions	Simple horizontal scaling; short expiry plus refresh token. **Status:** Proposed; retained only while JWT authentication remains part of the SRS and the team intends to implement it.

3.6 Technology Stack & Data Stores

Area	Updated technology
Frontend	React, Vite, JavaScript, CSS
Backend	Node.js, Express, REST/JSON
Database	MySQL — schema and integration pending
Authentication	Planned; JWT and password hashing if retained in the SRS
Testing	Vitest for frontend; Node.js built-in test runner for current backend endpoint tests
Version Control and CI	Git, GitHub and GitHub Actions

The technology table will be adjusted as the project develops. TypeScript, CSS modules, PostgreSQL, Prisma/Sequelize, Jest, Cypress and JMeter, listed in version 1.0, are not part of the current implementation unless the team adopts and uses them.

Planned Data Entities

The entities below are planned. They are not actual database tables until the team creates and tests the MySQL schema.

Entity	Key fields
users	id, name, email (unique), password_hash, role,

Entity	Key fields
	failed_attempts, locked_until, active
projects	id, name, owner_id, archived, created_at
project_members	project_id, user_id, member_role
tasks	id, project_id, title, description, due_date, priority, status, category, assignee_id, created_by, completed_at, deleted_at
tags / task_tags	id, name / task_id, tag_id
comments	id, task_id, author_id, body, created_at
activity_log	id, task_id, user_id, action, old_value, new_value, created_at
audit_log	id, user_id, event_type, ip, created_at
reminders	id, task_id, user_id, send_at, channel, sent

3.7 Risks & Mitigations

Risk	Mitigation
Database schema and integration are delayed	Assign ownership, define the schema milestone and review progress.
Frontend currently uses in-memory state	Integrate the frontend with the Express API and persistent MySQL storage.
Backend endpoints are incomplete	Implement and test required task and project endpoints incrementally.
Planned security controls are not implemented	Add authentication, authorization, validation and security testing before release.
Database outage or data loss	Daily backups, restore drills, connection retry with pooling
Slow queries as task volume grows	Indexes on project_id, assignee_id, status, due_date; pagination
Reminder emails not delivered	Retry queue with back-off; in-app reminder as fallback
Security vulnerabilities (injection, XSS, broken access control)	Validation, parameterised queries, server-side RBAC, ZAP scans in CI

Risk	Mitigation
Scope creep in a short project	Prioritised requirements (High first); Low items deferred

3.8 Traceability to Requirements

All components in the right-hand column are planned unless stated in Section 3.4.

Requirement	Component(s)
TM-F-001 to 004 (Authentication)	Auth Controller, Auth Service, Notification Service
TM-F-010 to 014 (Task management)	Task Controller, Task Service, Notification Service
TM-F-020 to 023 (Projects, search, reminders)	Project Service, Task Service, Notification Service
TM-F-030 to 032 (Kanban, dashboard, history)	Task Board / Dashboard UI, Report Service
TM-F-040 to 043 (Comments, admin, export, access)	Task Service, Auth Service, Report Service, Project Service
TM-NF-001, 005 (Performance, scalability)	Indexed DB, stateless API instances, pagination
TM-NF-002, 007 (Availability, recovery)	Deployment monitoring, backup jobs
TM-SR-001 to 007 (Security)	Gateway/TLS, Auth Service, JWT middleware, validation layer, audit log

3.9 Security Architecture

Threat modelling with STRIDE. The mitigations below are planned controls and are not yet implemented.

Threat	Example	Mitigation (planned)
Spoofing	Stolen or guessed credentials	Password hashing, lock-out after 5 failures, rate limiting, short-lived JWT
Tampering	Modifying another user's task via API	Server-side RBAC and project-membership checks, input validation
Repudiation	User denies deleting a task	Activity log and audit log with user, time, IP
Information disclosure	Sniffing traffic, leaking passwords in logs	TLS 1.2+, no secrets in logs, generic error messages
Denial of service	Flooding login or API	Rate limiting, request size limits, pagination

Threat	Example	Mitigation (planned)
Elevation of privilege	Member calls admin endpoint	Role claims verified on each request; admin routes separated

4. Design

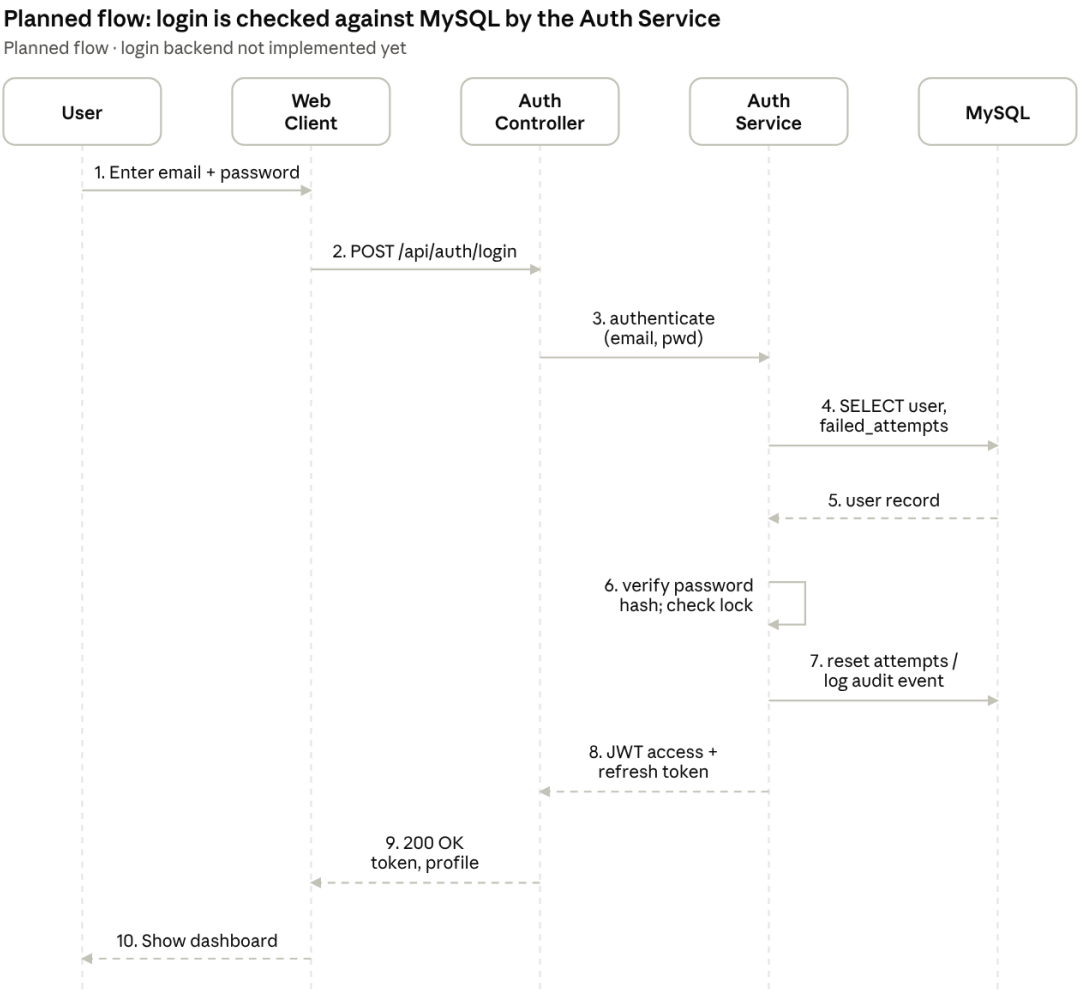
4.1 Design Overview

The backend is designed as layered modules: controllers (HTTP concerns), services (business rules), and repositories (data access). Dependencies point downward only, so each service can be unit-tested with mocked repositories. The frontend is organised by feature with a shared API client.

Current status: the React + Vite frontend is implemented, and the basic Express server is implemented. Service and database integration remains pending.

4.2 UML Sequence Diagrams

Two key flows document the intended architecture: user login (authentication) and creation of a task with reminder scheduling. Both are **planned flows**; the login and reminder backend features are not implemented yet. The planned task-creation sequence is retained, but its database insert and reminder-scheduling steps must not be presented as working functionality.



login sequence · 5 participants, planned flow

Figure 2 - Sequence Diagram: User Login (Planned Flow)

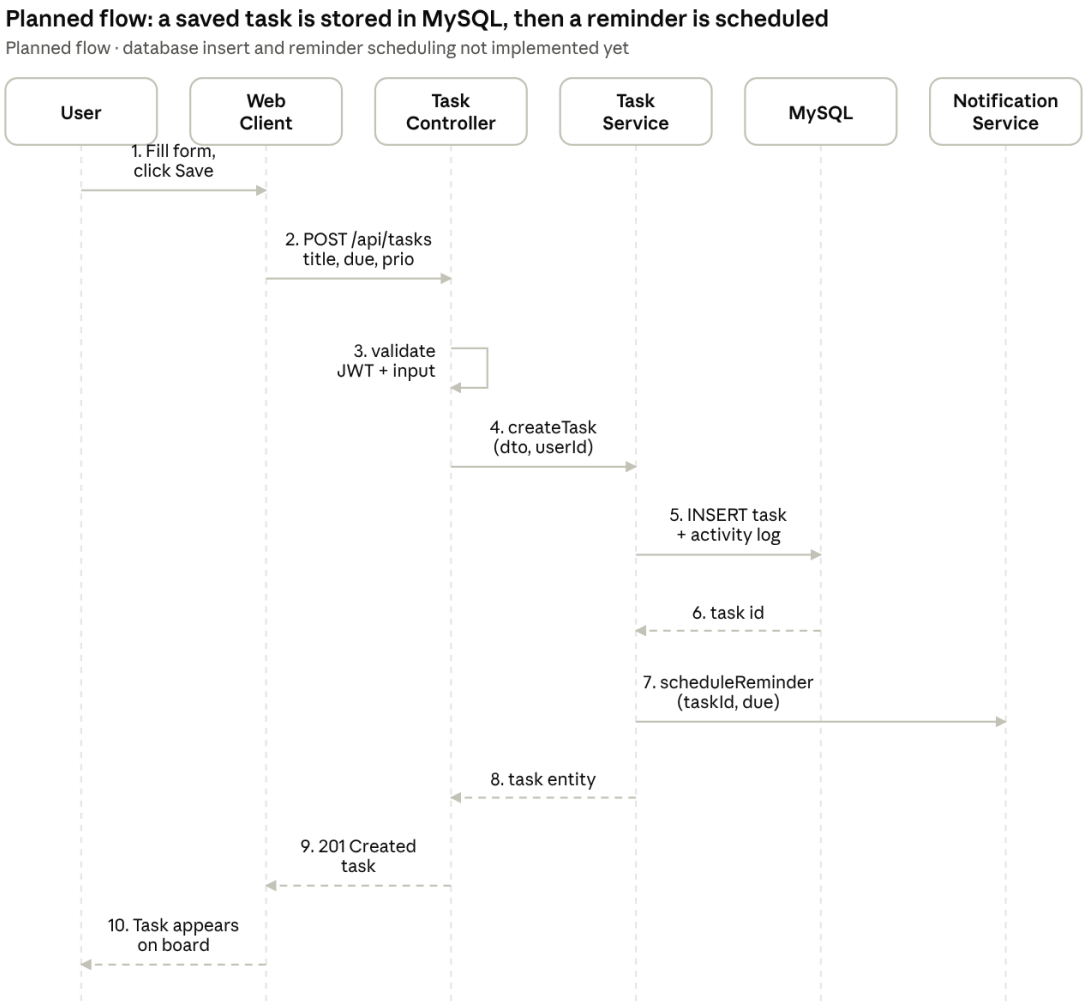


Figure 3 - Sequence Diagram: Create Task and Schedule Reminder (Planned Flow)

4.3 API Design

The current Express API returns JSON responses. HTTPS and authentication requirements will be enforced as part of deployment and security implementation. Once authentication is implemented, protected endpoints will require the header Authorization: Bearer <token>.

Implementation Status

Endpoint	Status
GET /	Implemented — returns the API welcome message
GET /health	Implemented — returns the health status

Endpoint	Status
POST /api/auth/register	Planned
POST /api/auth/login	Planned
POST /api/tasks	Planned
GET /api/tasks	Planned
PATCH /api/tasks/:id	Planned
DELETE /api/tasks/:id	Planned
GET /api/reports/dashboard	Planned

The current backend tests cover the root and health endpoints. Planned endpoints are not marked as implemented until they exist and their tests pass. The remaining planned endpoints in the tables below (logout, reset, comments, CSV export) are likewise Planned.

Component 1 - Auth API (Planned)

Endpoint	Method	Request	Response	Errors	Status
/api/auth/register	POST	{name, email, password}	201 {userId}	400 Invalid data, 409 Email exists	Planned
/api/auth/login	POST	{email, password}	200 {accessToken, refreshToken, profile}	401 Invalid credentials, 423 Account locked, 429 Too many requests	Planned
/api/auth/logout	POST	{refreshToken}	204	401 Unauthorised	Planned
/api/auth/reset	POST	{email} or {token, newPassword}	202 / 200	400 Invalid or expired token	Planned

Component 2 - Task API (Planned)

Endpoint	Method	Request	Response	Errors	Status
/api/tasks	POST	{projectId, title, description, dueDate, priority,	201 {task}	400 Validation error, 401, 403 Not a member	Planned

Endpoint	Method	Request	Response	Errors	Status
		assigneeId?}			
/api/tasks?status=&priority=&q=&sort=	GET	Query parameters	200 {items[], page, total}	401, 403	Planned
/api/tasks/{id}	PATCH	Changed fields, e.g. {status:"DONE"}	200 {task}	400, 403, 404 Not found	Planned
/api/tasks/{id}	DELETE	-	204 (soft delete)	403, 404	Planned
/api/tasks/{id}/comments	POST	{body}	201 {comment}	400, 403, 404	Planned
/api/reports/dashboard	GET	projectId (optional)	200 {countsByStatus, percentComplete, overdue}	401, 403	Planned
/api/tasks/export.csv	GET	Same filters as list	200 text/csv	401, 403	Planned

4.4 Error Handling, Logging & Monitoring

- **Implemented:** Express JSON middleware, root endpoint, health endpoint.
- **Planned:** Standardised error responses ({code, message, details?}), structured logging with request IDs, rate limiting, authentication monitoring and production monitoring (uptime checks, error rate, response-time percentiles, failed-login rate and reminder-delivery failures).

Request logging, security monitoring and standardised error handling are not claimed as existing until verified in the code.

4.5 UX Design

Implemented: the React/Vite dashboard, task creation, task status changes, filtering/search, deletion and task counts.

Planned: drag-and-drop Kanban, responsive refinements, accessibility improvements (WCAG 2.1 AA) and dark mode.

4.6 Open Issues & Next Steps

1. Complete the MySQL schema and database setup documentation.
2. Integrate the Express backend with MySQL.

3. Implement task CRUD endpoints and their tests.
4. Connect the React frontend to the API.
5. Implement the remaining SRS features, including authentication, projects, reminders, dashboards and collaboration.
6. Continue testing and update the SAD as implementation progresses.

Future enhancements, if they remain outside the agreed scope: recurring tasks, file attachments, calendar integration and WebSockets.

5. Appendices

5.1 Glossary: SPA, REST, JWT, RBAC, ORM, ADR, STRIDE, TLS, CRUD.

5.2 References: IEEE 42010, OWASP Top 10 / ASVS, NIST SP 800-160, Task Manager SRS v1.0.

5.3 Tools: PlantUML, draw.io, Swagger / OpenAPI, Postman, GitHub.